Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	R S Kirithic	
Register Number	24011101082	

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:
- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:
- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer
- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

The dataset was loaded directly from the pickled batch files (`data_batch_1` to `data_batch_5` for training and `test_batch` for testing) already stored in Google Drive from the previous experiment, rather than downloaded again. Each batch file unpickles to a dictionary whose `data` field is a 10000×3072 array of pixel values and whose `labels` field is a list of 10000 class indices; this was reshaped into $10000 \times 32 \times 32 \times 3$ images before use.

8. Experimental Procedure and Results

Task 1: Dataset Preparation

CIFAR-10 was loaded from Drive and the pixel values were normalised to $[0, 1]$, confirmed by a pixel range of $[0.00, 1.00]$. The training set had 50,000 images and the test set had 10,000 images, each $32 \times 32 \times 3$, across the 10 usual classes. Ten sample images are shown in Figure 1, and the class distribution of the training set is shown in Figure 2.

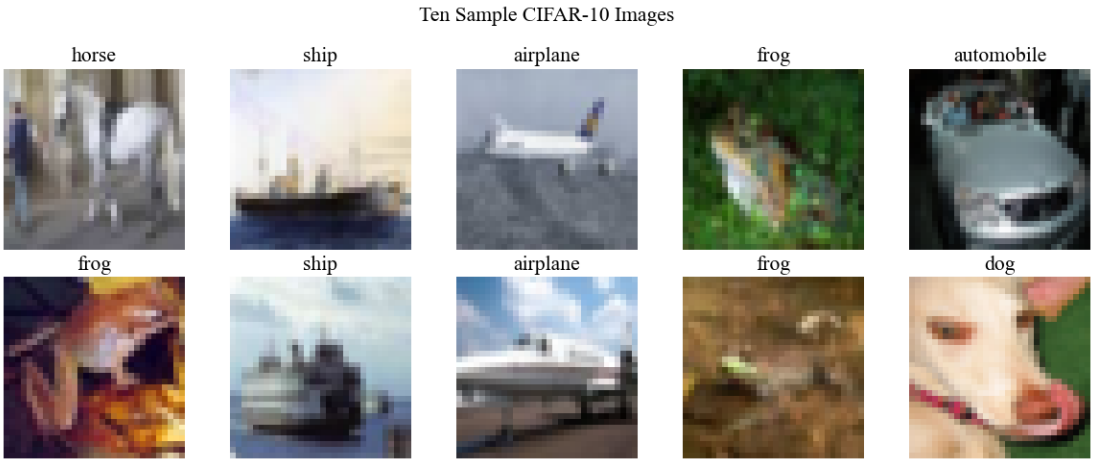

Figure 1: Ten Sample CIFAR-10 Images

Figure 2: Class Distribution (Training Set)

Observation: The training set has exactly 5,000 images in each of the 10 classes, so it is perfectly balanced and does not need any class weighting or resampling before training.

Task 2: Convolution Layer — Kernel Size Comparison

A single-filter convolution with 'valid' padding and stride 1 was applied to one sample image using 3x3, 5x5 and 7x7 kernels.

Kernel Size	Output Shape	Expected Size (Formula)
3×3	30×30	30
5×5	28×28	28
7×7	26×26	26

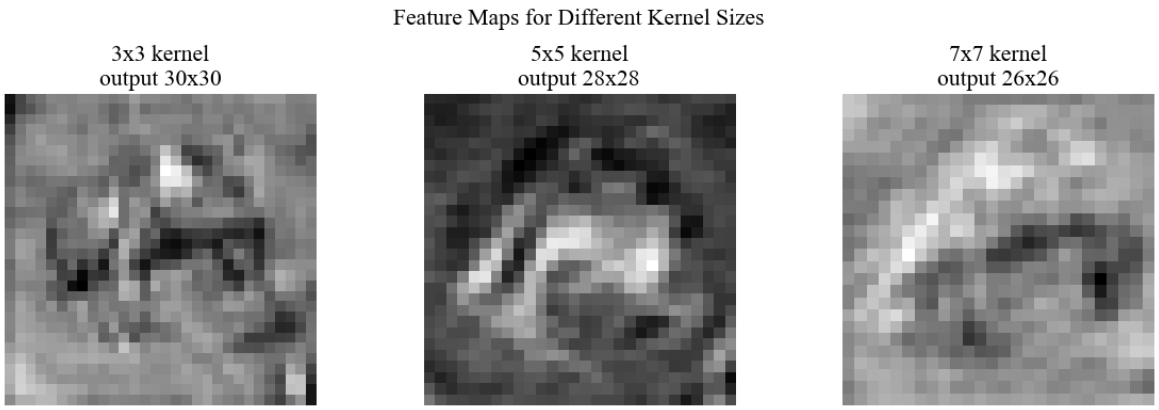

Figure 3: Feature Maps for Different Kernel Sizes

Observation: All three output sizes match the $\frac{N-F+2P}{S} + 1$ formula from Section 3 exactly, using $N = 32$, $P = 0$, $S = 1$. Larger kernels lose more pixels around the border, so the output feature map shrinks as the kernel size increases.

Task 3: Stride and Padding

The same demonstration image and a fixed 3x3 kernel were used to compare stride 1 against stride 2, and 'same' padding against 'valid' padding.

Stride	Padding	Output Shape
1	Valid	30×30
2	Valid	15×15
1	Same	32×32
2	Same	16×16

Observation: 'Same' padding keeps the output the same size as the input when the stride is 1, while 'valid' padding always shrinks it. Increasing the stride from 1 to 2 roughly halves the output size in both padding modes, which again matches the formula from Section 3.

Task 4: Feature Map Visualization

A freshly initialized (untrained) Conv2D layer with 16 filters was applied to a sample image, and 8 of the resulting feature maps are shown in Figure 3.

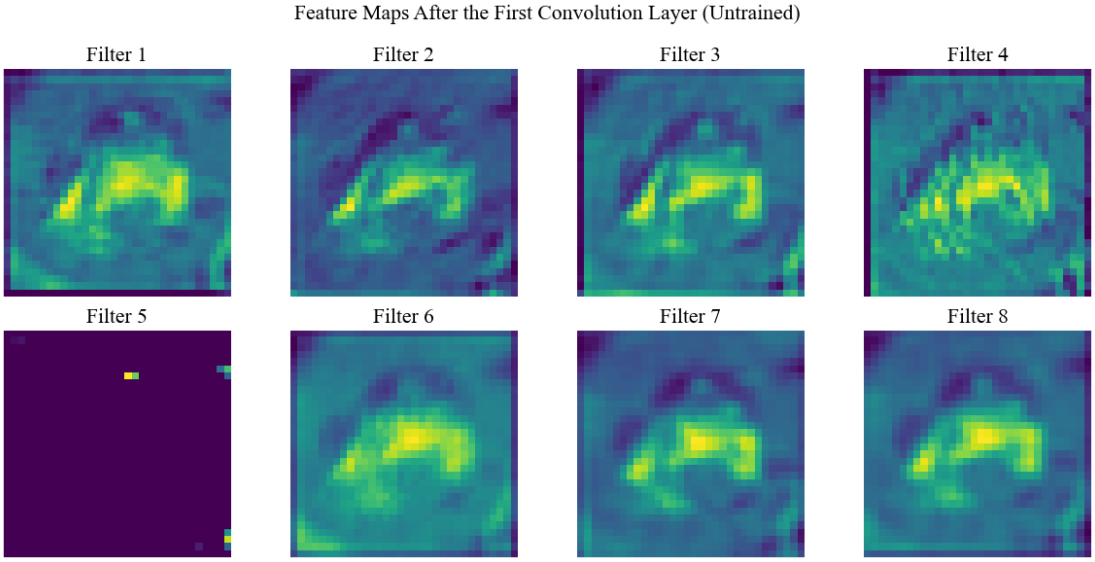

Figure 4: Feature Maps After the First Convolution Layer (Untrained)

Observation: Since the filters are randomly initialised and not yet trained, most of them respond to roughly the same regions of the image, mainly the brighter areas around the frog's body. Filter 5 stays almost completely dark, which happens when a filter's random weights combine with the ReLU activation in a way that zeroes out nearly its entire output for this particular image – a filter can end up doing essentially nothing until training adjusts its weights.

Task 6: Building and Training the Main CNN

The specified architecture, Input → Conv(16, 3x3) → ReLU → MaxPool → Conv(32, 3x3) → ReLU → MaxPool → Flatten → Dense(10) → Softmax, was trained with the Adam optimizer, 20 epochs and batch size 32, on 45,000 training images with 5,000 held out for validation, on the Google Colab GPU runtime. The model had 25,578 parameters in total. Training took 115.95 seconds. By the last epoch, training accuracy reached 76.61% and validation accuracy reached 66.40%.

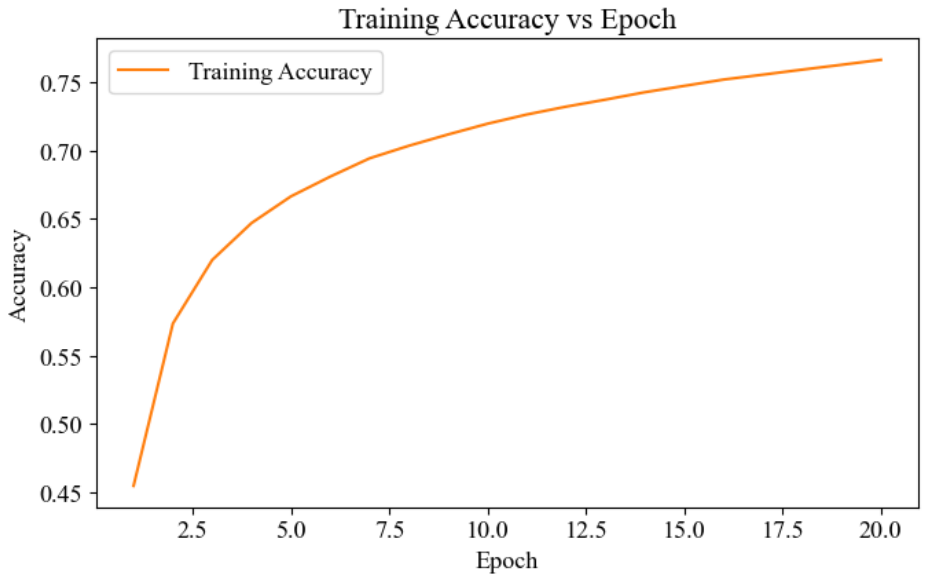

Figure 5: Training Accuracy vs Epoch

Observation: Training accuracy rises quickly over the first few epochs and then continues climbing more slowly, reaching 76.61% by epoch 20 without flattening out completely, which suggests a few more epochs could still improve it slightly.

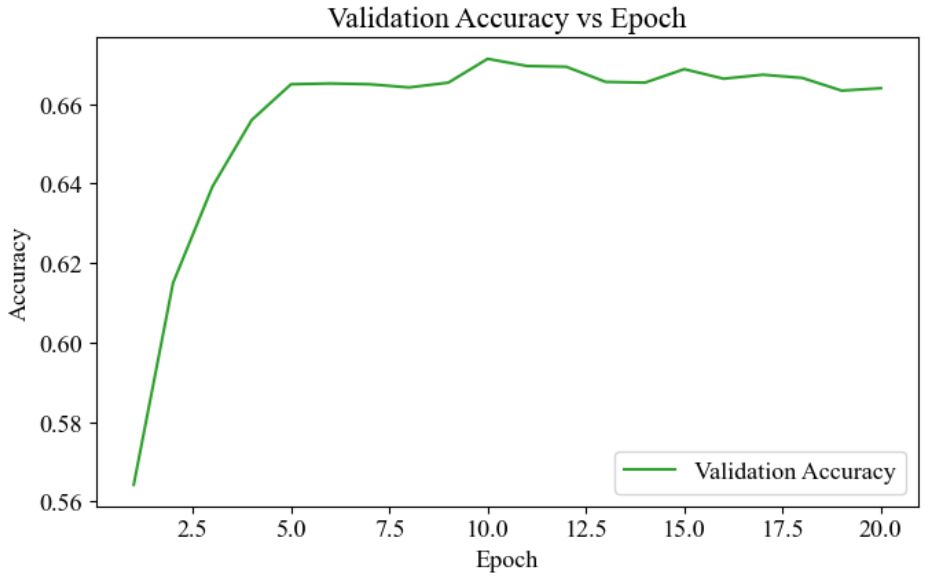

Figure 6: Validation Accuracy vs Epoch

Observation: Validation accuracy rises quickly at first, then levels off around 66–67% from about epoch 9 onward while training accuracy keeps increasing. This growing gap between training and validation accuracy (76.61% against 66.40% by the last epoch) is a sign of mild overfitting, which is expected here since this small network has no dropout or other regularization.

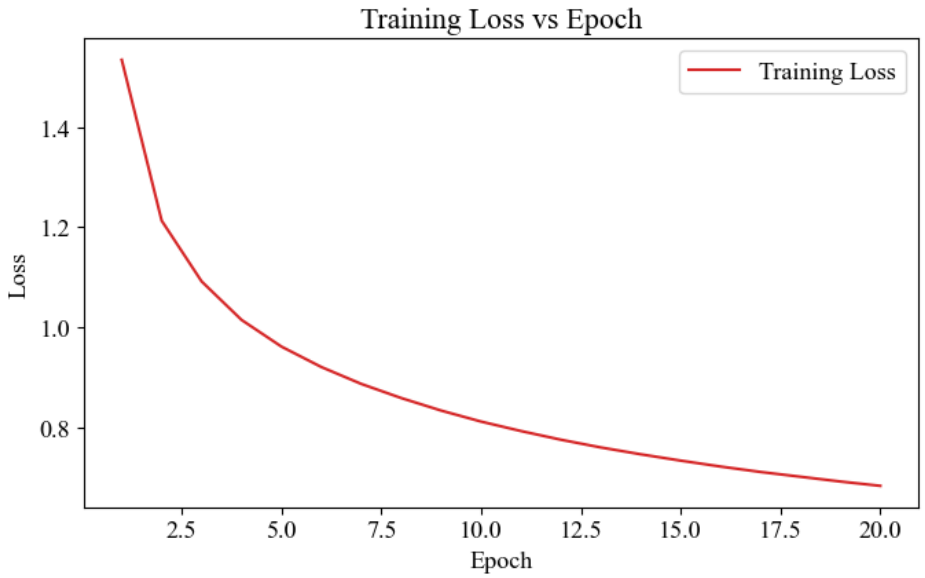

Figure 7: Training Loss vs Epoch

Observation: Training loss decreases steadily and smoothly across all 20 epochs, from about 1.53 to 0.68, with no bumps.

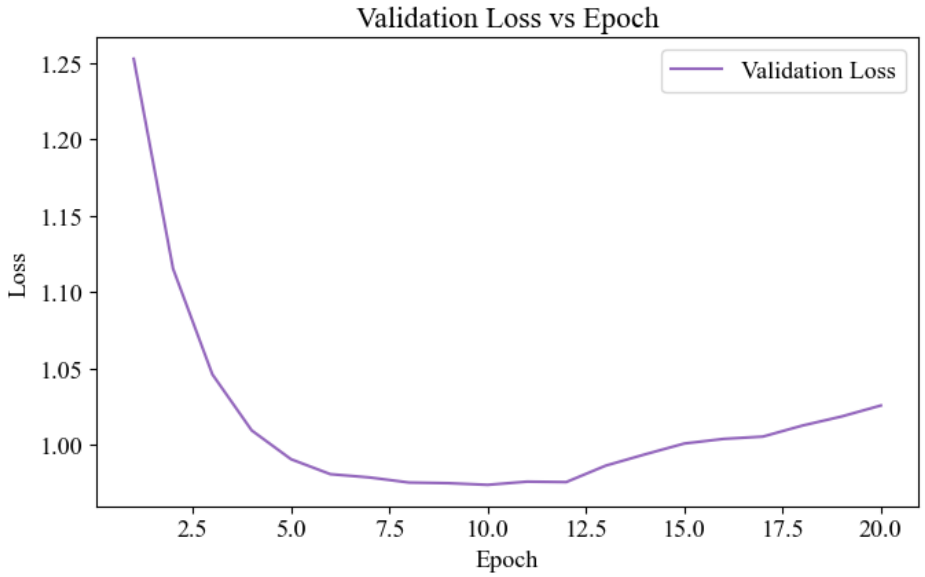

Figure 8: Validation Loss vs Epoch

Observation: Validation loss drops quickly for the first few epochs, reaches its lowest point around epoch 9–10, and then rises slowly for the rest of training even as training loss keeps falling – the same overfitting pattern seen in the accuracy curves.

Task 7: Model Evaluation

The trained model was evaluated on the full 10,000-image test set.

Metric	Value
Testing Accuracy	65.84%
Weighted Precision	66.77%
Weighted Recall	65.84%
Weighted F1-score	66.04%

Class	Precision	Recall	F1-score	Support
Airplane	0.68	0.71	0.69	1000
Automobile	0.83	0.74	0.78	1000
Bird	0.60	0.50	0.55	1000
Cat	0.43	0.53	0.47	1000
Deer	0.59	0.62	0.60	1000
Dog	0.58	0.55	0.57	1000
Frog	0.81	0.66	0.73	1000
Horse	0.67	0.75	0.71	1000
Ship	0.72	0.81	0.76	1000
Truck	0.77	0.72	0.74	1000
Accuracy	0.66			10000
Macro Avg	0.67	0.66	0.66	10000
Weighted Avg	0.67	0.66	0.66	10000

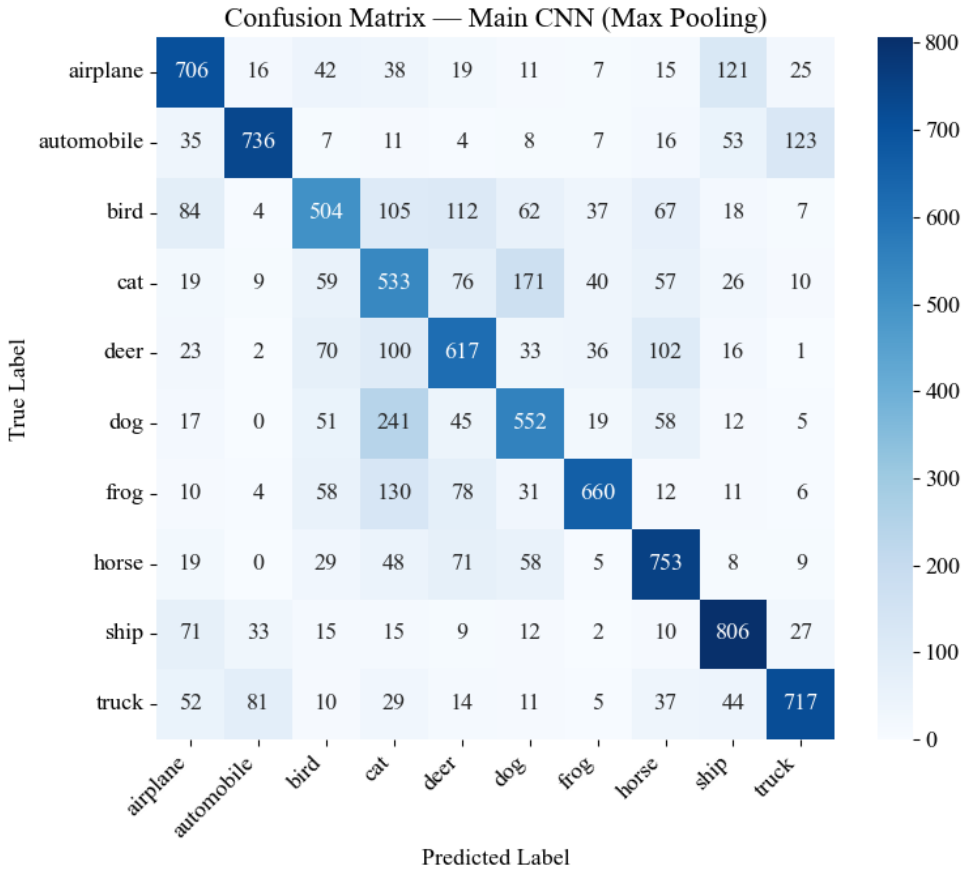

Figure 9: Confusion Matrix – Main CNN (Max Pooling)

Observation: The largest confusion is between cat and dog: 171 cats were predicted as dog and 241 dogs were predicted as cat, matching cat's precision of only 0.43, the lowest in the table. Automobile and truck are also mixed up in both directions (123 automobiles predicted as truck, 81 trucks predicted as automobile), and airplane and ship show some mutual confusion as well (121 airplanes predicted as ship, 71 ships predicted as airplane) – in each case the confused classes share a similar overall shape or background at this resolution.

9. Mandatory Plots

All of the plots required for this experiment – sample images, class distribution, training accuracy, validation accuracy, training loss, validation loss, feature maps and the confusion matrix – are presented above in Section 8, alongside the task they belong to, each with its own brief observation.

10. Additional Exercises

Exercise 1: Output Size for a 64x64 Image, 5x5 Kernel, Stride 2, Padding 2

Using the formula from Section 3 with $N = 64$, $F = 5$, $S = 2$, $P = 2$,

$$\left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = 32$$

This was checked against an actual convolution layer (zero-padded by 2, then a 5×5 kernel with stride 2), which produced a 32×32 output, confirming the calculation.

Exercise 2: Trainable Parameters for 64 Filters, 3x3 Kernel, RGB Input

$$(3 \times 3 \times 3 + 1) \times 64 = 1792$$

This was also checked against an actual Conv2D layer with 64 filters, a 3×3 kernel and a 3-channel input, which reported 1792 trainable parameters, confirming the calculation.

Exercise 3: ReLU vs Sigmoid Activation

The convolutional layers of the main CNN were switched from ReLU to Sigmoid activation, keeping everything else identical, and trained for the same 20 epochs.

Activation	Test Accuracy	Training Time (s)
ReLU (main CNN)	65.84%	115.95
Sigmoid	60.04%	101.52

Observation: ReLU reached noticeably higher accuracy than Sigmoid under the same settings (65.84% against 60.04%). By the last epoch the Sigmoid network was still climbing steadily (training accuracy 60.93%, well below ReLU's 76.61%), which is consistent with Sigmoid activations being more prone to small gradients that slow down learning, especially in the earlier layers of the network.

Exercise 4: Replace Max Pooling with Average Pooling

This is the same comparison as Task 5, so the two pooling types are compared once, together, below.

Task 5 / Exercise 4: Max Pooling vs Average Pooling

The main CNN already is the max-pooling version of this comparison. A second copy of the same architecture, with 'MaxPooling2D' replaced by 'AveragePooling2D' everywhere, was trained under identical settings.

Pooling	Test Accuracy	Output Shape After 1st Pool	Training Time (s)
Max	65.84%	16×16	115.95
Average	67.38%	16×16	98.47

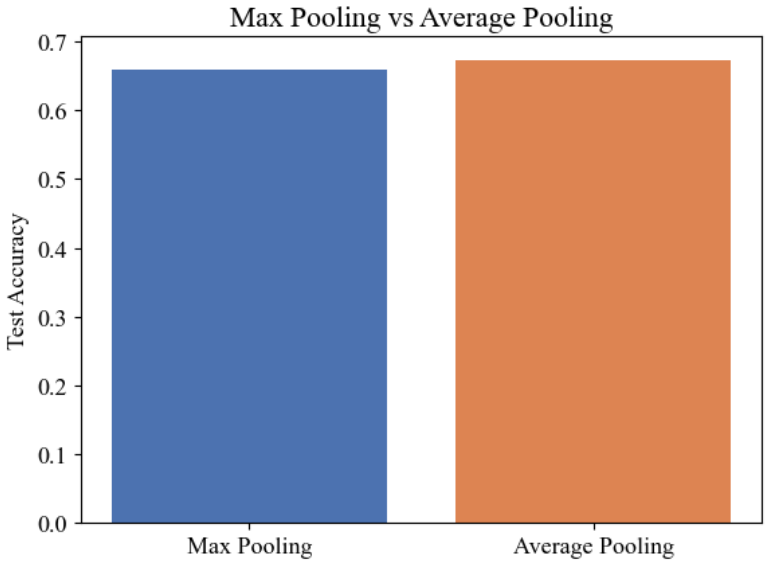

Figure 10: Max Pooling vs Average Pooling

Observation: Both pooling types reduce the spatial size in exactly the same way, from 32×32 down to 16×16 after the first pooling layer, since the output size only depends on the pooling window and stride, not on whether the maximum or the average is taken. Average pooling reached slightly higher test accuracy than max pooling here (67.38% against 65.84%) and also trained a little faster. This is not guaranteed to hold in general – max pooling usually keeps the strongest activations, which can help when the most distinctive feature in a region matters most, while average pooling smooths over the whole region, which can help when the useful information is spread out rather than concentrated at a single point.

Exercise 5: Increasing Filters from 16 to 64

The first convolution layer's filter count was increased from 16 to 64, keeping the second layer at 32 filters, and trained under identical settings.

First-Layer Filters	Parameters	Test Accuracy	Training Time (s)
16 (main CNN)	25,578	65.84%	115.95
64	40,746	67.05%	131.77

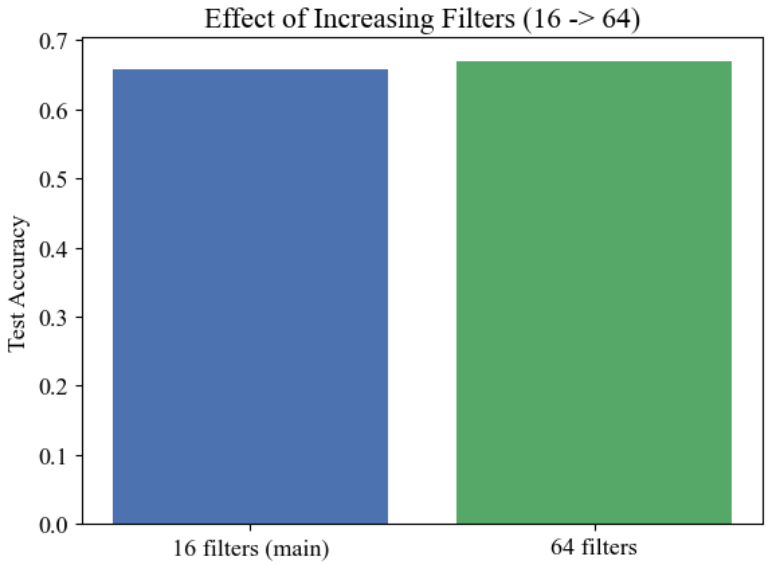

Figure 11: Effect of Increasing Filters (16 to 64)

Observation: Increasing the first layer's filter count from 16 to 64 gave a small accuracy improvement (65.84% to 67.05%) at the cost of more parameters (25,578 to 40,746) and a longer training time (115.95 seconds to 131.77 seconds). The accuracy gain here is modest compared to the added computation, which suggests this particular network was not strongly limited by the number of first-layer filters to begin with.

11. Results

Metric	Value
Training Accuracy	76.61%
Testing Accuracy	65.84%
Precision	66.77%
Recall	65.84%
F1-score	66.04%
Number of Parameters	25,578

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?** A fully connected layer treats every pixel as an independent input and learns a separate weight for each one, which for a $32 \times 32 \times 3$ image alone already means thousands of weights per neuron, and it throws away the spatial relationship between neighbouring pixels entirely. Convolution instead slides a small kernel across the image, so the same small set of weights is reused at every position, giving far fewer parameters and letting the network learn local patterns, such as edges and textures, that are meaningful regardless of where they appear in the image.
2. **How does stride affect the feature map size?** Stride controls how many pixels the kernel moves between each application. A stride of 1 slides the kernel one pixel at a time and produces the largest possible output, while a larger stride skips more pixels and shrinks the output size roughly in proportion, as shown directly in Task 3, where changing the stride from 1 to 2 roughly halved the output size in both padding modes.
3. **What is the role of padding?** Padding adds extra pixels, usually zeros, around the border of the input before convolution. Without it ('valid' padding), the output shrinks with every

convolution layer and pixels near the edges are used in fewer computations than pixels near the centre. 'Same' padding is chosen so that the output has the same spatial size as the input, which was confirmed directly in Task 3.

4. **Why is pooling used?** Pooling reduces the spatial size of the feature maps, which cuts down the number of parameters and computation needed in the layers that follow, and it also makes the network's output a little less sensitive to small shifts or distortions in exactly where a feature appears in the image. Task 5 showed this reduction directly: both max and average pooling brought the feature map down from 32×32 to 16×16 after the first pooling layer.
5. **How do feature maps represent image characteristics?** Each feature map is the result of one filter being applied across the whole image, so it shows where and how strongly that particular pattern (an edge at some orientation, a texture, a corner, and so on) is present. Early layers tend to pick up simple, low-level patterns like edges and colour contrasts, which is consistent with what Task 4 showed for an untrained layer – most filters responding to the same bright regions of the sample image, since at that point they only reflect random initial weights rather than anything learned yet.
6. **Why do CNNs require fewer parameters than MLPs?** An MLP's first layer would need a separate weight for every pixel in the image for every neuron in that layer, so the parameter count grows directly with the image size. A CNN's convolution layer instead has one small kernel per filter, reused at every spatial position across the whole image, so the parameter count depends only on the kernel size and the number of filters, not on the image's width or height. Task 6's main CNN illustrates this directly: with two small convolution layers it has only 25,578 parameters in total, most of which actually come from the final dense layer rather than the convolutions themselves.

13. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. Simon Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. TensorFlow Documentation: `https://www.tensorflow.org`
5. Alex Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (CIFAR-10 dataset documentation): `https://www.cs.toronto.edu/~kriz/cifar.html`